



SAP KDT 심화 과정 2기

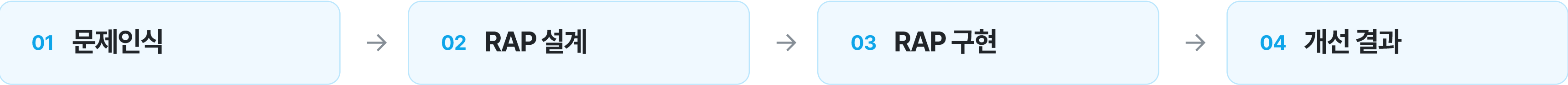
Portfolio

Cloud ABAP · RAP Project

박가을 010-9744-3565 · everjoy15388@gmail.com

프로젝트 개요

SAP KDT 심화 과정 2기 · B07 박가을



PROJECT INFO

프로젝트명	SAP Clean Core 기반 S-Mobile MM-SD-FI 통합 RAP 애플리케이션 구축
담당	박가을(B07) · 담당 [00]공통 범위부터 ~[06]구매오더까지
대표 프로그램	06. 구매오더관리 — Classic ABAP 병행 개발로 AS-IS/TO-BE 직접 비교
개발기간	2026.08.12 ~ 09.09 (전체 일정 2026.07.20 ~ 09.14)

CONTENTS

- 01 회사 개요 및 조직 구조
- 02 AS-IS → TO-BE
- 03 RAP 설계
- 04 RAP 구현
- 05 대표 프로그램 상세 — 구매오더
- 06 개발한 전체 프로그램
- 07 마무리

회사 개요 및 조직 구조

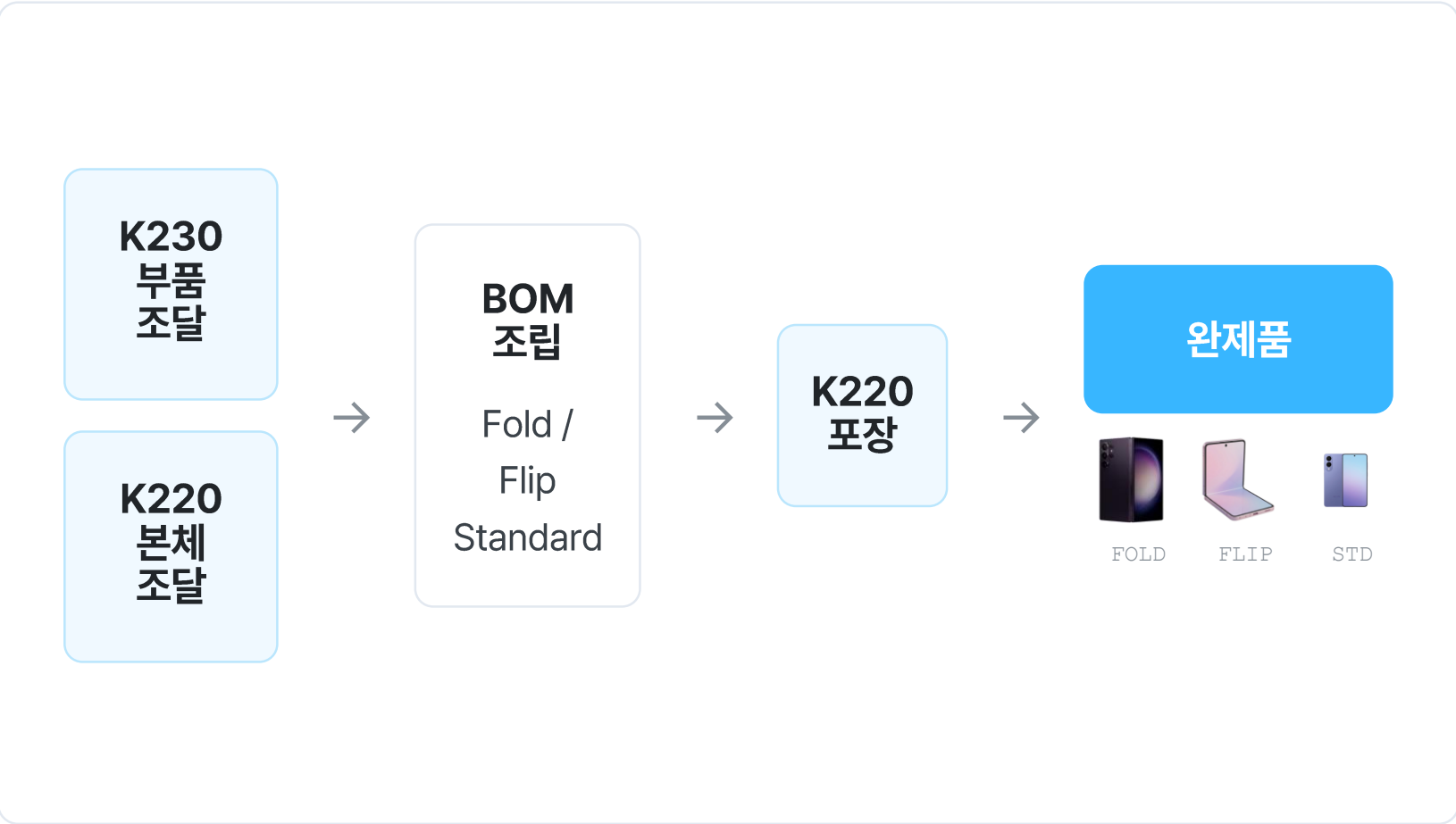


S-Mobile은 신제품 스마트폰 3종(Fold, Flip, Standard) 출시를 앞둔 가상 기업이다.
반제품·부품·포장재를 모두 외부에서 조달해 조립·포장까지 수행하는 유통·조립 기업으로, 별도의 원자재 가공 공정은 갖고 있지 않다.
자재 등록부터 구매·입고·판매·출고·회계 처리까지의 업무가 부서별 시스템으로 나뉘어 데이터 불일치와 비효율이 발생해왔고, 이를 하나의 SAP RAP 기반 통합 시스템으
로 구축하는 것이 이번 프로젝트의 출발점이다.

ORGANIZATION

구분	값	설명
회사코드	K200	가상 기업 S-Mobile Company
플랜트	K220	본체 조달 및 포장
플랜트	K230	부품·전자 자재 조달
구매조직	1000	Basic Purchasing Org
구매그룹	001	전자부품(PCB·본체) 담당
구매그룹	002	케이블·소모품 담당
구매그룹	003	포장재 담당

MATERIAL FLOW



Classic ABAP에서 RAP으로 — 왜 전환이 필요한가

01 채번을 프레임워크가 대신해주지 않는다

UUID나 문서번호를 자동 발급하는 표준 기능이 없어 저장 로직을 처음부터 끝까지 직접 설계해야 한다. 이번 구매오더 Classic 클론에서도 저장·삭제 로직은 아직 골격 수준에 머물러 있다.

02 서치헬프를 화면마다 손으로 만든다

필드 하나에 서치헬프를 붙이려면 이벤트 핸들러와 팝업 화면을 별도로 구현해야 한다. 벤더 목록 하나를 쓰기 위해 별도의 ALV 화면과 이벤트 처리 클래스를 새로 만들어야 했다.

03 임시저장 개념이 없다

작업 중인 데이터를 안전하게 보관하려면 별도의 임시 테이블과 화면 상태 관리를 처음부터 설계해야 한다.

04 화면 흐름을 전부 손으로 설계한다

아직 뜨지 않은 화면의 값을 참조하는 순환 의존 버그가 발생하기 쉽다. 실제로 화면 간 참조 순서 문제로 화면이 뜨지 않는 오류를 겪었다.

RAP은 이 네 가지를 **Managed 채번 · CDS Value Help · Draft · Determination/Validation**으로 대체한다.

데이터 모델 — 담당 모듈 00~06 테이블 관계도

담당 범위 7개 모듈은 공통 자재(마스터)를 중심으로 구매정보레코드와 구매오더까지 하나의 참조 체계로 연결된다. 회계 계정과 계정결정 규칙은 별도 축으로 존재하며, 벤더 마스터의 결제 계정을 통해 FI 축과 연결된다.

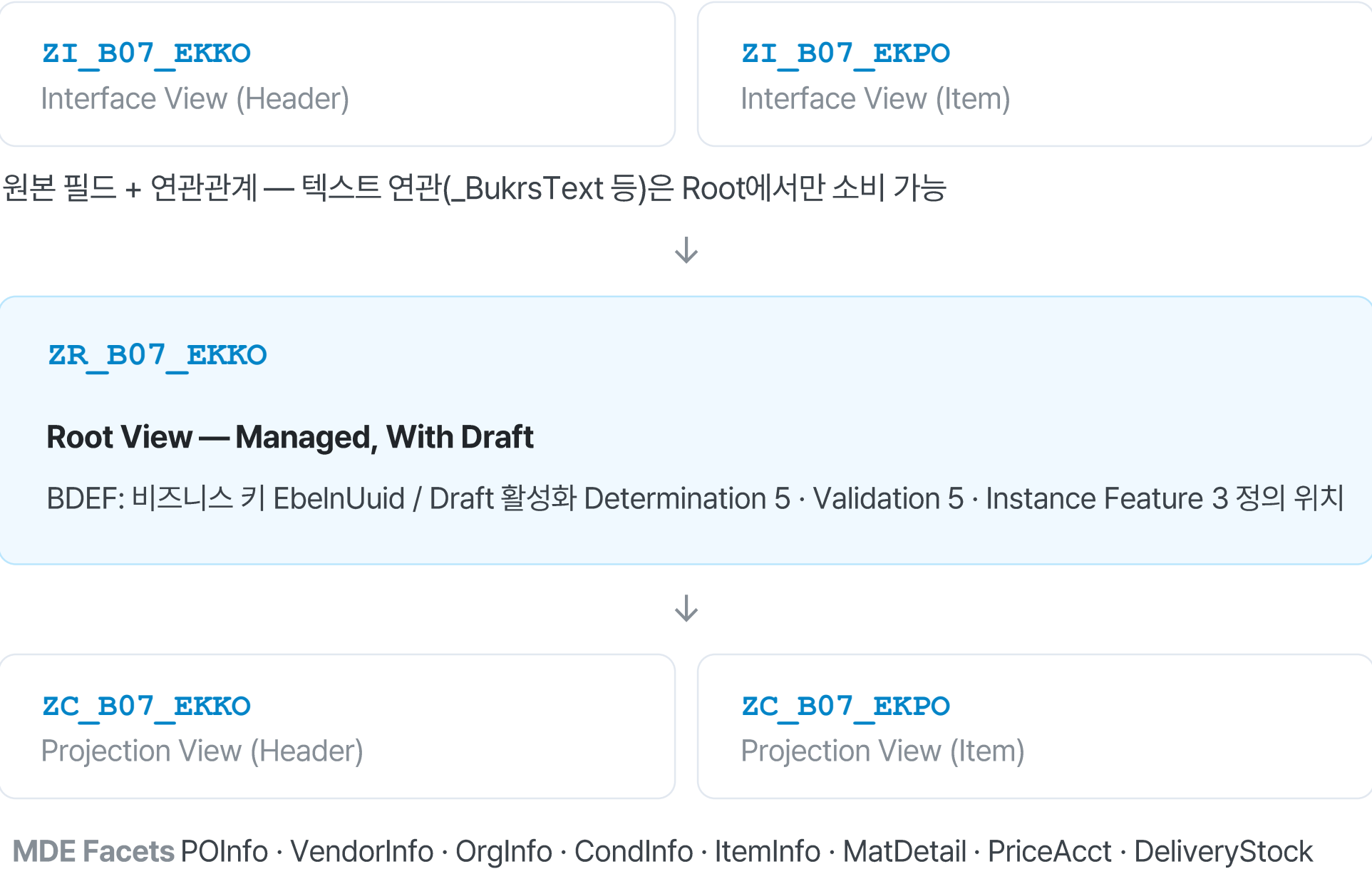
TABLES

모듈	테이블	주요 필드	역할
00	공통 Search Help	—	12개 F4 CDS
01	ZTB07MARA	mat_uuid(PK), Mtart, Bklas	자재 마스터
02	ZTB07SKA1	saknr(PK), Glact	FI 계정 마스터
03	ZTB07LFA1	lif_uuid(PK), Akont	공급업체 마스터
04	ZTB07T030	Bwart+Ktosl(PK), Saknr	회계계정결정
05	ZTB07EINA / EINE	inf_uuid(PK)	구매정보레코드
06	ZTB07EKKO / EKPO	ebeln_uuid(PK)	구매오더 헤더/아이템

RELATIONS

ZTB07LFA1.Akont → ZTB07SKA1.Saknr 공급업체 → 매입채무 계정
ZTB07T030.Saknr → ZTB07SKA1.Saknr 계정결정 → FI 계정
ZTB07EINA.lif_uuid → ZTB07LFA1 PIR → 공급업체
ZTB07EINA.mat_uuid → ZTB07MARA PIR → 자재
ZTB07EINE.inf_uuid → ZTB07EINA 구매조건 → PIR 헤더
ZTB07EKKO.lif_uuid → ZTB07LFA1 PO 헤더 → 공급업체
ZTB07EKPO.ebeln_uuid → ZTB07EKKO PO 아이템 → 헤더
ZTB07EKPO.mat_uuid → ZTB07MARA PO 아이템 → 자재
ZTB07EKPO.inf_uuid → ZTB07EINA PO 아이템 → PIR(선택)

Interface에서 Consumption까지, 구매오더 오브젝트 체인



SEARCH HELP VIEWS · 7

Search Help View	대상 필드
ZI_B07_EBELN_F4	Ebeln
ZI_B07_BSART_F4	Bsart
ZI_B07_INFNR_F4	Infnr
ZI_B07_INSMK_F4	Insmk
ZI_B07_EPSTP_F4	Epstp
ZI_B07_MWSKZ_F4	Mwskz
ZI_B07_POSTAT_F4	Postat

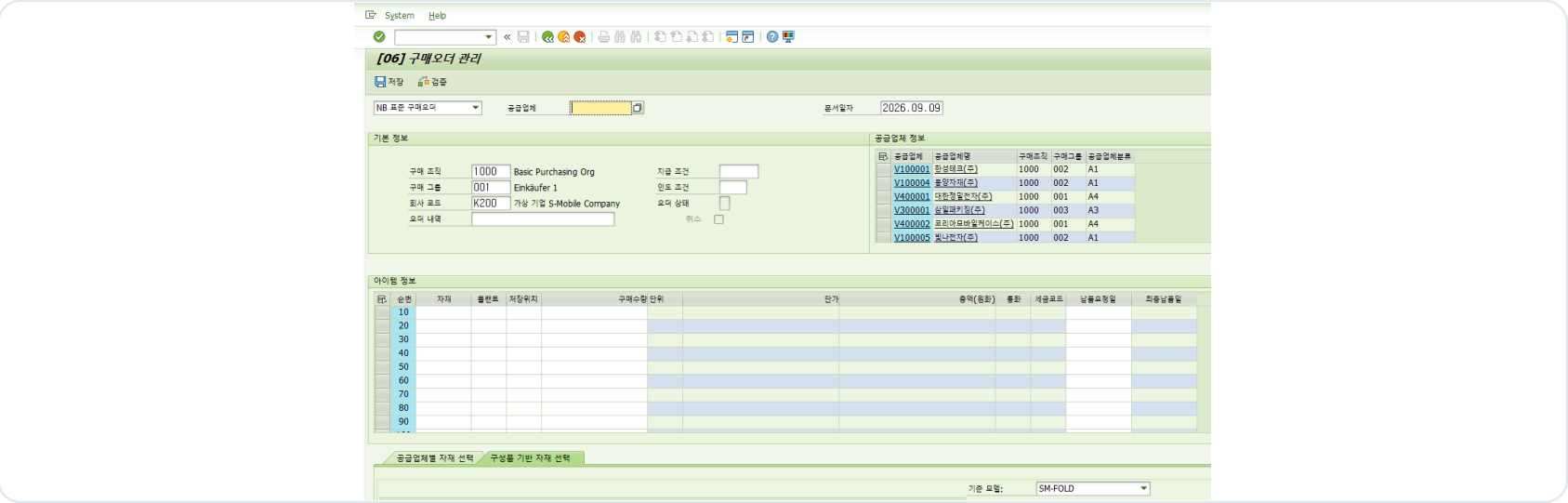
Epstp(품목범주)는 도메인 고정값이 아니라 소스 테이블 T163Y 로 확정 — 디버깅으로 직접 확인

Root 하나에 검증·기본값·화면제어 로직이 몰려 있는 구조 — 전체 매핑은 Slide 10에서 이어짐

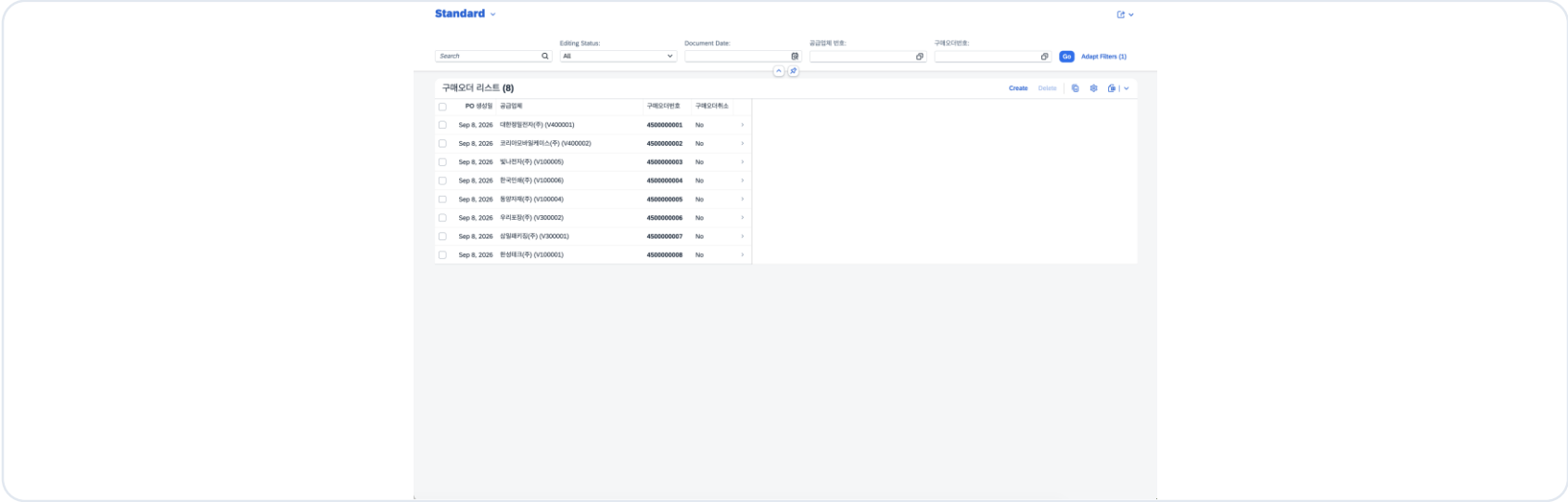
같은 업무, 다른 구현 — 오브젝트 단위 비교

한계 영역	Classic ABAP 구현	RAP 구현
채번	SNRO 넘버레인지 객체를 화면 PAI에서 수동 GET_NUMBER 호출	ZNRB07_EBE 넘버레인지 + Determination(SetEbelnNumber)이 cl_numberrange_runtime 자동 호출
서치헬프	lcl_event_handler 클래스 + 별도 팝업 화면(F4) 이벤트를 필드마다 구현	@Consumption.valueHelpDefinition으로 CDS Value Help 선언, Search Help View 7개로 일괄 처리
임시저장	별도 임시 테이블·화면 상태 관리 미구현(골격 단계에 머뭇)	Managed Draft — Edit / Activate / Discard / Resume / Prepare 표준 액션
검증 로직 위치	화면 100~300번대 PAI 곳곳에 분산, 화면 진입 순서에 의존	Root BDEF에 Validation 5개로 중앙집중, 트리거 시점을 프레임워크 선언으로 명시
필드 기본값	화면 진입 시점에 코드로 직접 SET	Determination이 필드 변경 이벤트에 반응해 자동 실행

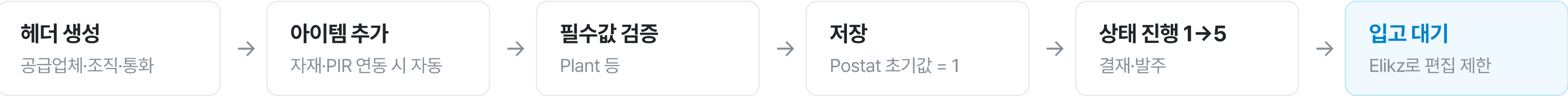
Classic ABAP 메인 화면 (벤더 리스트)



RAP List Report 화면 (구매오더 8건)



대표 프로그램 — PO 생성부터 입고 대기까지



POSTAT 상태값

상태	의미
1	서비스오더 미생성
2	서비스오더 생성
3	결재승인 및 발주
4	입고가능
5	종료

List Report 초기 화면

Standard

Document Date: 구매오더번호:

구매오더 리스트

PO 생성일	공급업체	구매오더번호	구매오더취소
To start, set the relevant filters and choose "Go".			

자재번호:
—
단가:
—

Create

Plant: *

Continue

Cancel

Plant 필수값 경고 모달

신규 PO 생성 화면

New: Purchase Order

구매오더 정보

공급업체 정보

구매 조직 정보

구매 조건 정보

아이템

PO 생성일: Sep 8, 2026

문서유형: 표준 구매오더 (NB)

구매오더내역: PCB 정기발주

구매오더번호: -

구매오더통화: KRW

구매오더취소: ☐

공급업체 정보

공급업체: V400001

공급업체명:

구매 조직 정보

회사코드:

구매조직:

구매그룹:

구매 조건 정보

지급조건: 0001

인코텀즈:

아이템

구매오더 아이템 리스트

아이템번호	자재번호	물량	수량	구매가격	입고완료
No items available.					

Draft updated

Create

Discard Draft

FS를 TS로 — 직접 내린 설계 판단

FS는 기능 요구사항만 정의하고 구현 방식은 열어둔 항목이 많았다. 아래는 그 항목을 RAP 오브젝트로 옮기며 직접 결정한 설계 판단이다.

FS 요구사항	TS 설계 판단
PO 생성일 = 현재일, 통화 = KRW	Determination(SetHeaderDefaults)에서 헤더 생성 시점에 자동 세팅, 이후 사용자가 수정 가능한 단순 기본값으로 설계
Ebeln은 자동 채번	Managed 채번 채택, 넘버레인지 객체 ZNRB07_EBE 지정 후 Determination(SetEbelnNumber)에서 cl_numberrange_runtime 호출
Item 필수값: Plant “입력 방식은 자유”	Plant는 자재마다 다를 수 있어 Header가 아닌 Item 단위 필수값으로 설계, 미입력 시 저장 시점에 경고 모달
Item 순번 10/20/30 자동	Early Numbering으로 Ebelp를 10 단위 증가 처리, Draft 상태에서 순번이 어긋나는 문제를 %is_draft 매핑 추가로 해결
Item 통화 = Header 값	Determination이 Item 생성 시 Header의 Waers를 그대로 상속하도록 설계
입고완료·IV완료·PO취소 시 필드 disabled	Instance Feature(Elikz 등)로 후속 프로세스 진행 여부에 따라 편집 가능 범위를 동적으로 제한

설계 총괄 - Root 하나에 담긴 15개 로직

Determination은 필드 기본값을 채우고, Validation은 저장 시점에 데이터를 검증하며, Instance Feature는 화면 편집 가능 범위를 제어한다. 세 종류는 트리거 시점이 서로 달라 같은 “로직”이라도 실행되는 순간이 다르다.

Determination · 6	트리거 시점	시나리오
SetHeaderDefaults	헤더 생성 시	—
SetVendorUuid	벤더코드 포커스아웃	①
SetEbelnNumber	헤더 저장 시(채번)	—
SetItemDefaults	아이템 생성 시	—
SetInfoRecordDefault	자재 입력 시(PIR 존재)	②
SetMaterialUuid	자재코드 입력 시	—

Instance Feature · 3	트리거 시점	시나리오
Loekz	Draft 편집 중 상시	—
Elikz	후속 프로세스 진행 후	—
Waers	저장 전후	③

Validation · 5	트리거 시점	시나리오
CheckRequired	저장 시(필수값)	—
CheckVendorValid	저장 시(벤더 유효성)	⑥
CheckOrgData	저장 시(조직 데이터)	④
CheckPositiveQty	저장 시(수량 양수)	⑤
CheckDuplicateItem	저장 시(아이템 중복)	—

Early Numbering · 1	트리거 시점	시나리오
Ebelp 자동채번	아이템 생성 시(10 단위)	—
시나리오 번호가 없는 항목도 실제로 구현·동작하는 로직이며, 다음 슬라이드부터는 화면 증빙이 확보된 6개 만 순서대로 심화한다.		

벤더코드 입력만으로 이름을 채운다

WHY

벤더코드(Lifnr)를 입력하면 벤더명이 즉시 채워지도록 설계했다. 저장 시점이 아니라 필드 변경 즉시 실행하는 이유는, 사용자가 입력을 마치기 전에 자신이 입력한 코드가 실제 존재하는 벤더인지 눈으로 바로 확인할 수 있게 하기 위함이다. 조회 조건은 LifUuid가 비어 있고 Lifnr이 입력된 경우로 한정해, 이미 UUID가 연결된 레코드를 매 변경마다 다시 조회하지 않도록 했다.

역할 분담 — 시나리오 ⑥ 예고

이 로직은 벤더 차단 여부(Loevm)를 조회하지 않는다. SELECT 절에 lif_uuid, lifnr, name1만 있고 loevm은 아예 없다. 이름 자동완성이라는 편의 기능과 차단 벤더 검증이라는 업무 규칙을 의도적으로 분리한 설계이며, 차단 검증은 저장 시점의 CheckVendorValid(시나리오 ⑥)가 담당한다.

Determination SetVendorUuid · lhc_ZR_B07_EKKO

```
METHODS SetVendorUuid FOR DETERMINE ON MODIFY
IMPORTING keys FOR zr_b07_ekko~SetVendorUuid.

METHOD SetVendorUuid.
DATA: lt_update TYPE TABLE FOR UPDATE zr_b07_ekko.

READ ENTITIES OF zr_b07_ekko IN LOCAL MODE
ENTITY zr_b07_ekko
FIELDS ( LifUuid Lifnr )
WITH CORRESPONDING #( keys )
RESULT DATA(lt_ekko).

SELECT lif_uuid, lifnr, name1 FROM ztb07lfa1
INTO TABLE @DATA(lt_db_lfa1).

LOOP AT lt_ekko INTO DATA(ls_ekko)
WHERE LifUuid IS INITIAL AND Lifnr IS NOT INITIAL.

READ TABLE lt_db_lfa1 INTO DATA(ls_db_lfa1) WITH KEY lifnr = ls_ekko-Lifnr.
IF sy-subrc = 0.
APPEND VALUE #( %tky      = ls_ekko-%tky
                  LifUuid = ls_db_lfa1-lif_uuid
                  Name1    = ls_db_lfa1-name1 ) TO lt_update.

ENDIF.
ENDLOOP.

IF lt_update IS NOT INITIAL.
MODIFY ENTITIES OF zr_b07_ekko IN LOCAL MODE
ENTITY zr_b07_ekko
UPDATE FIELDS ( LifUuid Name1 )
WITH lt_update.
ENDIF.
ENDMETHOD.
```

벤더코드 하나로 이름까지

공급업체 코드 V400001을 입력하고 포커스를 벗어나면, 별도 조회 없이 공급업체명 “대한정밀전자(주)”가 즉시 채워진다.

Before – 벤더코드만 입력

공급업체 정보

공급업체:
V400001

공급업체명:
대한정밀전자(주)

Supplier	Name 1	Planning Group	Short Description
V400001	대한정밀전자(주)	A4	Major vendors

Show All Items

After – 벤더명 자동 채움

공급업체 정보

공급업체:
대한정밀전자(주) (V400001)

공급업체명:
대한정밀전자(주)

구매정보레코드 번호 하나로 자재정보를 끌어온다

WHY DELETE lt_ekpo WHERE Infnr IS INITIAL 로 PIR을 참조하지 않는 아이템은 대상에서 제외해, 마스터에 없는 스팟성 구매 품목은 자동 로직을 타지 않고 수동 입력을 허용한다. 단가는 EINE(플랜트별 구매조건)을 Werks 기준으로 조회하고, 해당 플랜트 조건이 없으면 기존 입력값을 유지하는 폴백을 뒀다.

자동 vs 수동 — 자동으로 채우는 필드는 MatUuid·Matnr·Meins·Netpr 뿐. 수량은 사용자가 직접 입력한다.

Determination SetInfoRecordDefault · lhc_zi_b07_ekpo

```
METHODS SetInfoRecordDefault FOR DETERMINE ON MODIFY
IMPORTING keys FOR ekpo~SetInfoRecordDefault.

METHOD SetInfoRecordDefault.
DATA: lt_update TYPE TABLE FOR UPDATE zi_b07_ekpo.

READ ENTITIES OF zr_b07_ekko IN LOCAL MODE
ENTITY Ekpo
  FIELDS ( Infnr Werks Netpr )
  WITH CORRESPONDING #( keys )
RESULT DATA(lt_ekpo).

DELETE lt_ekpo WHERE Infnr IS INITIAL.
CHECK lt_ekpo IS NOT INITIAL.

SELECT inf_uuid, infnr, mat_uuid FROM ztb07eina
FOR ALL ENTRIES IN @lt_ekpo
WHERE infnr = @lt_ekpo-Infnr
INTO TABLE @DATA(lt_db_eina).

SELECT inf_uuid, werks, netpr FROM ztb07eine
FOR ALL ENTRIES IN @lt_db_eina
WHERE inf_uuid = @lt_db_eina-inf_uuid
INTO TABLE @DATA(lt_db_eine).

SELECT mat_uuid, matnr, meins FROM ztb07mara
FOR ALL ENTRIES IN @lt_db_eina
WHERE mat_uuid = @lt_db_eina-mat_uuid
INTO TABLE @DATA(lt_db_mara).
```

```
LOOP AT lt_ekpo INTO DATA(ls_ekpo).
  READ TABLE lt_db_eina INTO DATA(ls_eina)
    WITH KEY infnr = ls_ekpo-Infnr.
  CHECK sy-subrc = 0.

  READ TABLE lt_db_mara INTO DATA(ls_mara)
    WITH KEY mat_uuid = ls_eina-mat_uuid.
  READ TABLE lt_db_eine INTO DATA(ls_eine)
    WITH KEY inf_uuid = ls_eina-inf_uuid
      werks      = ls_ekpo-Werks.

  APPEND VALUE #( %tky      = ls_ekpo-%tky
                  MatUuid = ls_eina-mat_uuid
                  Matnr   = ls_mara-matnr
                  Meins    = ls_mara-meins
                  Netpr    = COND #( WHEN sy-subrc = 0
                                      THEN ls_eine-netpr
                                      ELSE ls_ekpo-Netpr ) )
    TO lt_update.
ENDLOOP.

IF lt_update IS NOT INITIAL.
  MODIFY ENTITIES OF zr_b07_ekko IN LOCAL MODE
  ENTITY Ekpo
    UPDATE FIELDS ( MatUuid Matnr Meins Netpr )
    WITH lt_update.
ENDIF.
ENDMETHOD.
```

참조 - 구매정보레코드 2026000001 마스터

자재명·단가 자동 채움, 수량은 수동 입력

2026000001

Edit

Delete

구매정보레코드 정보

공급업체 상세정보

자재 상세정보

아이템

구매정보 번호:
2026000001

구매정보 내역:
신규 업체(시험중)

구매단위:
EA

레코드 유형:
Standard (0)

구매정보 취소:
No

공급업체 상세정보

공급업체:
대한정밀전자(주) (V400001)

공급업체명:
대한정밀전자(주)

구매조직:
-

구매그룹:
-

자재 상세정보

자재:
PCB Assembly (PCB)

자재명:
PCB Assembly

아이템

구매정보 아이템 리스트 (1)

플랜트	플랜트명	구매가격	
부산 자재 창고 (K230)	부산 자재 창고	258,000 KRW	>

자재 상세정보

구매정보번호:

2026000001

수량:

2,000

EA

자재번호:

PCB

단가:

258,000

KRW

저장 전엔 입력 가능, 저장 후엔 읽기전용

WHY Determination이 값을 채우고 Validation이 값을 검증한다면, Instance Feature는 필드를 “편집할 수 있는가”를 동적으로 제어한다. 통화(Waers)는 채번 이후 회계·구매 정합성이 걸리므로 잠근다. 판단 기준은 저장 여부가 아니라 **Ebeln 채번 여부**다.

Instance Feature Waers·Loekz · lhc_ZR_B07_EKKO (Header)

```
METHOD get_instance_features.
  READ ENTITIES OF zr_b07_ekko IN LOCAL MODE
    ENTITY zr_b07_ekko
      FIELDS ( Ebeln )
      WITH CORRESPONDING #( keys )
      RESULT DATA(lt_ekko).

  result = VALUE #( FOR ls_ekko IN lt_ekko
    ( %tky      = ls_ekko-%tky
      %delete = COND #( WHEN ls_ekko-Ebeln IS NOT INITIAL
        THEN if_abap_behv=>fc-o-disabled
        ELSE if_abap_behv=>fc-o-enabled )
      %action-SetDeletionFlag
        = COND #( WHEN ls_ekko-Ebeln IS NOT INITIAL
          THEN if_abap_behv=>fc-o-enabled
          ELSE if_abap_behv=>fc-o-disabled )
      " 통화(Waers)는 PO 채번 (=최초 저장) 전까지만 편집 가능
      %field-Waers
        = COND #( WHEN ls_ekko-Ebeln IS NOT INITIAL
          THEN if_abap_behv=>fc-f-read_only
          ELSE if_abap_behv=>fc-f-unrestricted )
      " Loekz: 미채번 시 readonly, 채번 후에만 편집 가능
      %field-Loekz
        = COND #( WHEN ls_ekko-Ebeln IS NOT INITIAL
          THEN if_abap_behv=>fc-f-unrestricted
          ELSE if_abap_behv=>fc-f-read_only ) ) ).

ENDMETHOD.
```

반대 패턴 — Loekz 구매오더취소 필드는 정반대 조건이다. 미채번 문서는 읽기전용, 채번 후에만 편집 가능하다. 아직 존재하지 않는 문서를 취소할 수 없다는 업무 규칙을 그대로 코드 조건으로 옮겼다.

Instance Feature Waers·Postat · lhc_zi_b07_ekpo (Item)

```
METHOD get_instance_features.
  " 입고완료(Elikz=X)된 아이템은 삭제 불가
  READ ENTITIES OF zr_b07_ekko IN LOCAL MODE
    ENTITY ekpo
      FIELDS ( Elikz )
      WITH CORRESPONDING #( keys )
      RESULT DATA(lt_ekpo).

  CHECK lt_ekpo IS NOT INITIAL.

  " 아이템이 실제 DB에 이미 존재하는지 조회
  SELECT ebeln_uuid, ebelp FROM ztb07ekpo
  FOR ALL ENTRIES IN @lt_ekpo
  WHERE ebeln_uuid = @lt_ekpo-EbelnUuid
    AND ebelp      = @lt_ekpo-Ebelp
  INTO TABLE @DATA(lt_db_ekpo).

  result = VALUE #( FOR ls_ekpo IN lt_ekpo
    ( %tky      = ls_ekpo-%tky
      %delete = COND #( WHEN ls_ekpo-Elikz = abap_true
        THEN if_abap_behv=>fc-o-disabled
        ELSE if_abap_behv=>fc-o-enabled )
      " Waers: 기존 저장분이면 읽기전용
      %field-Waers = COND #( WHEN line_exists( ... )
        THEN if_abap_behv=>fc-f-read_only
        ELSE if_abap_behv=>fc-f-unrestricted )
      " Postat: 신규 생성 시 readonly - Waers와 반대
      %field-Postat = COND #( WHEN line_exists( ... )
        THEN if_abap_behv=>fc-f-unrestricted
        ELSE if_abap_behv=>fc-f-read_only ) ) ).

ENDMETHOD.
```

아이템 레벨의 같은 사고방식 Waers는 DB에 이미 저장된 아이템이면 읽기전용, 신규면 편집 가능. Postat는 그 반대 — 정식 저장된 문서에 대해서만 진행상태를 관리하겠다는 판단이다.

채번 전후로 바뀌는 편집 가능 범위

채번 전에는 통화 필드를 자유롭게 입력할 수 있지만, 최초 저장으로 문서번호가 채번된 이후에는 같은 필드가 회색 읽기전용으로 바뀐다.

Before - 채번 전, 편집 가능

구매오더 정보 공급업체 정보 구매 조직 정보 구매 조건 정보 아이템

PO 생성일:
Sep 8, 2026

문서유형:

구매오더번호:
-

구매오더통화:
KRW

After - 채번 후, 읽기전용

구매오더 정보 공급업체 정보 구매 조직 정보 구매 조건 정보 아이템

PO 생성일:
Sep 8, 2026

문서유형:

구매오더번호:
0000000000

구매오더통화:
South Korean Won (KRW)

조직 데이터는 SAP 표준 커스터마이징 테이블로 검증한다

WHY 회사코드(Bukrs)·구매조직(Ekorg)·구매그룹(Ekgrp)을 각각 T001 · T024E · T024에 존재하는 값인지 저장 시점에 검증한다. 입력 도중이 아니라 저장 시점인 이유는, 조직 데이터는 중간에 매번 막을 필요 없이 최종 커밋 시점의 정합성만 보장하면 되기 때문이다. 존재 여부만 필요하므로 TRANSPORTING NO FIELDS 로 불필요한 필드 조회 없이 확인한다.

범위 확인 — 이 Validation은 존재 여부만 확인하고 오류를 반환할 뿐 값을 바꾸지 않는다. 화면에서 관찰된 구매그룹 001 자동 표시는 이 코드가 한 일이 아니며, Value Help 등 별도 UI 동작으로 추정되나 코드로 확정하지는 못했다.

Validation CheckOrgData · lhc_ZR_B07_EKKO (Bukrs 블록, Ekorg·Ekgrp 동일 패턴)

```
METHODS CheckOrgData FOR VALIDATE ON SAVE
  IMPORTING keys FOR zr_b07_ekko~CheckOrgData.

METHOD CheckOrgData.
  READ ENTITIES OF zr_b07_ekko IN LOCAL MODE
    ENTITY zr_b07_ekko
      FIELDS ( Bukrs Ekorg Ekgrp )
      WITH CORRESPONDING #( keys )
    RESULT DATA(lt_ekko).

  SELECT bukrs FROM t001 INTO TABLE @DATA(lt_t001).
  SELECT ekorg FROM t024e INTO TABLE @DATA(lt_t024e).
  SELECT ekgrp FROM t024 INTO TABLE @DATA(lt_t024).

  LOOP AT lt_ekko INTO DATA(ls_ekko).
    IF ls_ekko-Bukrs IS NOT INITIAL.
      READ TABLE lt_t001 TRANSPORTING NO FIELDS
        WITH KEY bukrs = ls_ekko-Bukrs.
```

```
IF sy-subrc <> 0.
  APPEND VALUE #( %tky = ls_ekko-%tky )
    TO failed-zr_b07_ekko.
  APPEND VALUE #(
    %tky = ls_ekko-%tky
    %element-Bukrs = if_abap_behv=>mk-on
    %msg = new_message(
      id = 'ZMSGE_B07' number = '020'
      v1 = 'Company Code' v2 = ls_ekko-Bukrs
      severity = if_abap_behv_message=>severity-error ) )
    TO reported-zr_b07_ekko.
  ENDIF.
ENDIF.

" Ekorg → t024e, Ekgrp → t024 동일 패턴 반복
ENDLOOP.
ENDMETHOD.
```

존재하지 않는 조직 코드는 저장 시점에 걸린다

회사코드 ZZZZ, 구매조직 9999처럼 커스터마이징 테이블에 없는 값을 입력하고 저장하면 필드별 오류 메시지가 표시된다.

잘못된 조직 데이터 입력

구매 조직 정보

회사코드:
ZZZZ

구매조직:
999

구매 조직 정보

Value "999" does not exist.

Value "ZZZZ" does not exist.

저장 시 오류 메시지

회사코드:
[<] [x]

구매조직:
999

Value "999" does not exist.

구매그룹 서치헬프 — 없는 값 검색 (참고용)

구매그룹:
fsdsdfsdfsdf

mber	Telephone	Extension
No data		
Show All I		

서치헬프 동작 — CheckOrgData Validation과 무관

Apply 시점이 아니라 저장 시점에 걸린다

WHY — Apply와 Save의 시점 차이

수량이 0 이하인지 검증하는 로직을 Item Apply가 아니라 SAVE 시점에 걸리도록 설계했다. RAP Draft 흐름에서 Apply는 아이템 팝업의 값을 Draft에 임시 반영하는 단계이고, Validation ON SAVE는 문서 전체를 최종 커밋하는 시점에만 실행된다.

그래서 수량을 0으로 둔 채 아이템을 만들어도 **Apply는 그대로 성공** 하고, 헤더에서 전체 저장을 실행하는 순간에야 오류가 반환된다. 입력 중간마다 막지 않고 **최종 커밋 시점에만 적합성을 강제** 하는 설계다.

Apply → Draft 반영



Save → Validation 실행

Validation CheckPositiveQty · lhc_zi_b07_ekpo

```
METHODS CheckPositiveQty FOR VALIDATE ON SAVE
IMPORTING keys FOR Ekpo~CheckPositiveQty.

METHOD CheckPositiveQty.
READ ENTITIES OF zr_b07_ekko IN LOCAL MODE
ENTITY Ekpo
    FIELDS ( Menge )
    WITH CORRESPONDING #( keys )
RESULT DATA(lt_ekpo).

LOOP AT lt_ekpo INTO DATA(ls_ekpo) WHERE Menge <= 0.
    APPEND VALUE #( %tky = ls_ekpo-%tky ) TO failed-ekpo.
    APPEND VALUE #(
        %tky = ls_ekpo-%tky
        %element-Menge = if_abap_behv=>mk-on
        %msg = new_message(
            id = 'ZMSGE_B07' number = '023'
            v1 = 'Quantity'
            severity = if_abap_behv_message=>severity-error ) )
    TO reported-ekpo.
ENDLOOP.
ENDMETHOD.
```

Apply는 통과, Save에서 막힌다

수량 0인 아이템도 Apply 단계에서는 그대로 생성된다. 헤더에서 전체 저장을 실행하는 순간에만 수량 오류가 표시된다.

Apply 성공 - 음수 수량 아이템 생성됨

플랜트	수량
K220	-5,000 EA

Save 시점 오류

< Error

✖

Quantity must be greater than 0.

이름은 보여주되, 저장은 막는다

WHY — 시나리오 ①과의 역할 분담

시나리오 ①의 SetVendorUuid는 차단 여부(Loevm)를 확인하지 않고 이름만 채운다. 이 Validation은 그 반대편 역할을 맡는다. 저장 시점에 LifUuid가 해석되지 않았거나 Loevm이 차단 상태인 벤더면 저장을 막는다.

두 로직을 나눈 이유는, 입력 중에는 어떤 벤더 코드를 넣었는지 이름으로 바로 확인할 수 있게 하되, **거래 가능 여부라는 업무 규칙은 최종 저장 시점에만 엄격하게 적용** 하기 위함이다.

① SetVendorUuid

입력 즉시 · 이름 자동완성 loevm 조회 없음

⑥ CheckVendorValid

저장 시점 · 업무 규칙 loevm 차단 검증

Validation CheckVendorValid · lhc_ZR_B07_EKKO

```
METHODS CheckVendorValid FOR VALIDATE ON SAVE
  IMPORTING keys FOR zr_b07_ekko~CheckVendorValid.

METHOD CheckVendorValid.
  READ ENTITIES OF zr_b07_ekko IN LOCAL MODE
    ENTITY zr_b07_ekko
      FIELDS ( LifUuid )
      WITH CORRESPONDING #( keys )
    RESULT DATA(lt_ekko).

  SELECT lif_uuid, loevm FROM ztb07lfa1
    INTO TABLE @DATA(lt_db_lfa1).

  LOOP AT lt_ekko INTO DATA(ls_ekko)
    WHERE LifUuid IS NOT INITIAL.
    READ TABLE lt_db_lfa1 INTO DATA(ls_db_lfa1)
      WITH KEY lif_uuid = ls_ekko-LifUuid.

    IF sy-subrc <> 0 OR ls_db_lfa1-loevm = abap_true.
      APPEND VALUE #( %tky = ls_ekko-%tky )
        TO failed-zr_b07_ekko.
      APPEND VALUE #(
        %tky = ls_ekko-%tky
        %element-LifUuid = if_abap_behv=>mk-on
        %msg = new_message(
          id = 'ZMSGE_B07' number = '020'
          v1 = 'Vendor' v2 = 'blocked or not found'
          severity = if_abap_behv_message=>severity-error ) )
        TO reported-zr_b07_ekko.
    ENDIF.
  ENDLOOP.
ENDMETHOD.
```

차단된 벤더, 이름은 뜨지만 저장은 안 된다

차단된 벤더 V100007을 입력하면 이름은 여전히 자동으로 채워진다. 저장을 시도하면 “Vendor blocked or not found” 오류가 표시된다.

차단 벤더 마스터 – 거래종료 Yes (V100007)

저장 시도 – 차단 오류

미래산업(주) (V100007)

EditDelete

공급업체 정보

계정 정보

구매 정보

기타 정보

공급업체 분류:
Domestic payments (A/P) (A1)

공급업체 번호:
미래산업(주) (V100007)

공급업체명:
미래산업(주)

계정 정보

계정 타입:
Balance Sheet Account (X)

계정:
Accounts Payable (2110000)

구매 정보

구매조직:
1000

구매그룹:
003

기타 정보

최소주문금액:
500,000 KRW

주문통화:
KRW

거래종료:
Yes

거래종료:
Yes

공급업체 정보

공급업체:
V100007

구매 조직 정보

회사코드:

<

×

Vendor blocked or not found is not a real account.

Classic ABAP — 자유로운 만큼, 모든 걸 직접 책임지는 구조

정직한 인정 — 강점

자재 조달 화면의 BOM 막대그래프는 기준모델 (Fold/Flip/Standard)을 드롭다운으로 전환하면 구성 부품이 즉시 다시 그려지도록 구현했다. 화면과 이벤트를 코드 수준에서 직접 제어할 수 있는 Module Pool 구조였기에 가능한 자유도였다.

그 자유도의 비용 — 실제로 겪은 구조적 문제 2건

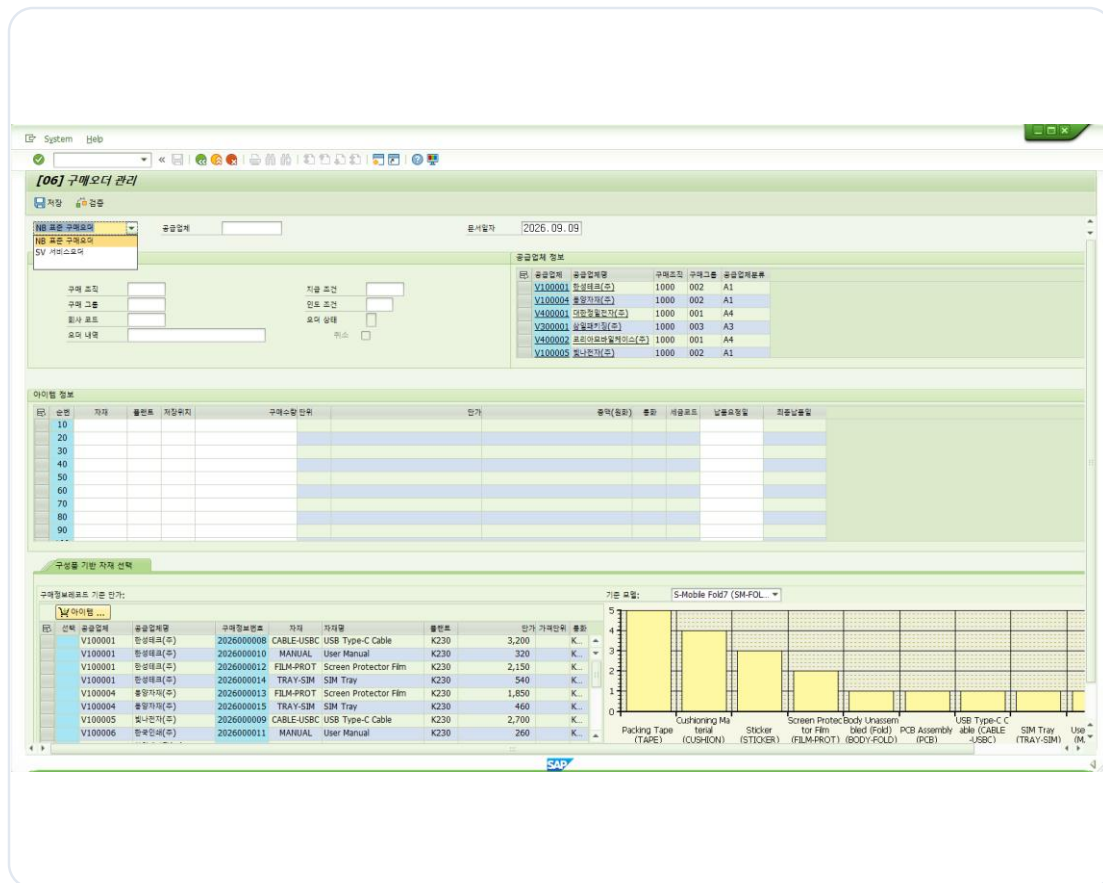
01 화면 103/104는 초기에 역방향 BOM 구조로 설계했다가 실제 조립 흐름과 맞지 않아 정방향으로 다시 설계했다. 설계 오류가 화면을 다 만든 뒤에야 드러났다.

02 `set_dynnr_tab`이 화면 103 활성화 뒤에야 채워지는 `gv_chart_show` 를 진입 조건으로 참조하면서, 화면 자체가 뜨지 않는 순환 의존 버그가 발생했다.

그럼에도 진행된 부분 — 현재 상태

ZMSG_B07 메시지 클래스에 오류 코드를 001~023, 100번대로 체계적으로 분리해 관리했고, 09월 9일 기준으로 BOM 화면에서 품목을 채운 뒤 발주를 확정하는 “발주 확인” 다이얼로그까지 동작한다. 초기 저장 로직이 골격 단계였던 시점보다 실제로는 더 진행된 상태다.

BOM 막대그래프 화면



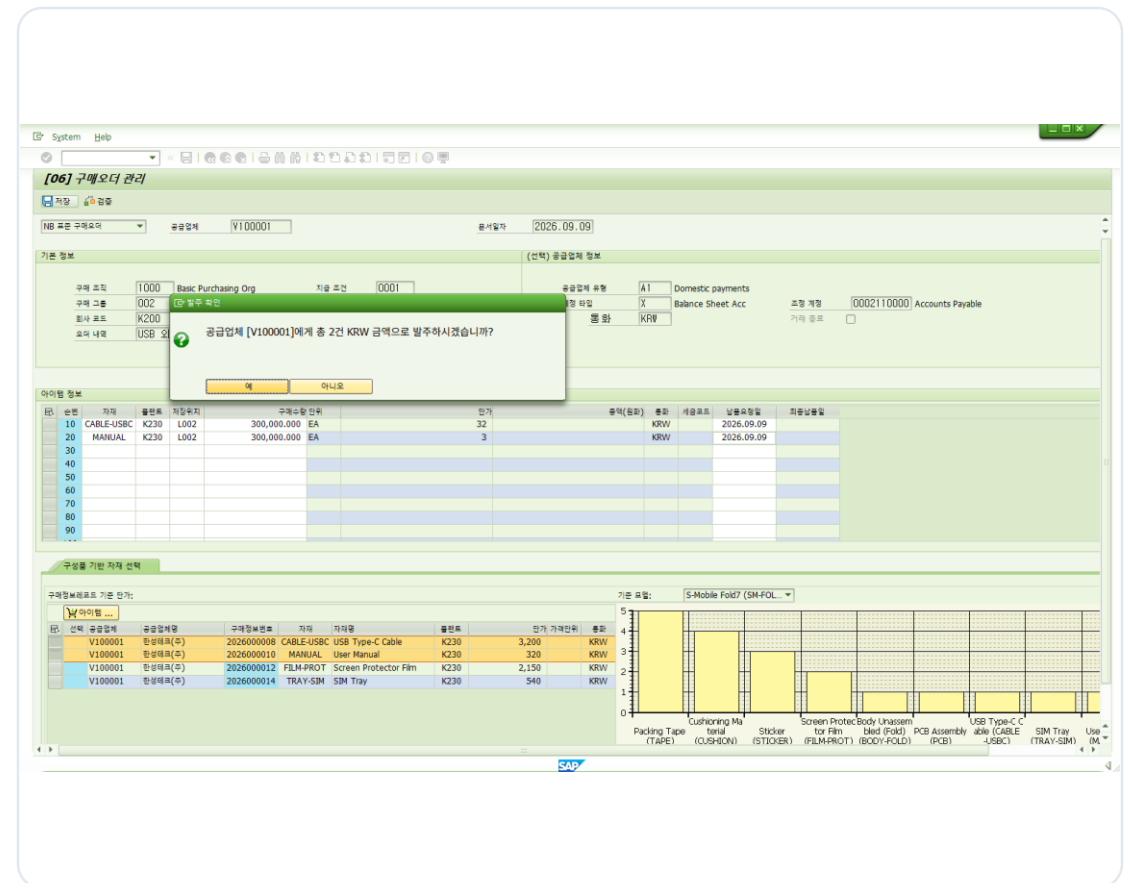
Module Pool 화면 103 · `set_dynnr_tab` (재구성)

```
" AS-IS — 화면 103 진입 조건이
"   화면 103에서만 채워지는 값을 참조
IF gv_chart_show = abap_true.
    set_dynnr_tab( 103 ).
ENDIF.
```

→ 화면 103 미진입,
`gv_chart_show` 는 계속 공백

```
" TO-BE — 진입 조건을 화면 상태와 분리
IF gv_tab_sel = 'TAB2'.
    set_dynnr_tab( 103 ).
ENDIF.
```

발주 확인 다이얼로그



RAP 전환으로 무엇이 달라졌는가

직접 설계해야 할 화면은 9개에서 2개로 줄고, 흩어져 있던 로직 15개가 Root BDEF 한 곳으로 모였다.

직접 설계한 화면 화면 9개 → List Report + Object Page	Classic		9개
	RAP		2개
한 곳에 모인 선언적 로직 화면 PAI 분산 → Root BDEF 집중	Classic		0개
	RAP		15개
임시저장 표준 액션 미구현 → Managed Draft 5종	Classic		0개
	RAP		5개
서치헬프 오브젝트 필드별 핸들러 → 공용 Search Help View	Classic		미집계
	RAP		7개

INSIGHT

Root BDEF 한 곳에 모인 15개 로직 —
Determination 6 · Validation 5 · Instance
Feature 3 · Early Numbering 1. 트리거 시점이 프레
임워크 규칙으로 고정되면서, 화면 순서에 의존하던 순
환 의존 버그가 구조적으로 발생하기 어려워졌다.

서치헬프의 Classic 측 수치는 필드별 개별 구현이라 집계
대상이 아니어서 표기하지 않았다.

버그가 사라진 게 아니라, 원인 파악 방식이 달라졌다

RAP 개발 중에도 버그는 있었다. Early Numbering에서 %is_draft 매핑을 빠뜨려 EN_NO_FAILED_NO_MAPPED 덤프가 난 적이 있다. 차이는 원인 파악 방식이다. Classic의 버그는 화면이 그냥 뜨지 않아 화면 흐름을 손으로 추적해야 했고, RAP의 버그는 ADT가 덤프 원인과 수정 지점을 바로 알려줬다.

항목	Classic ABAP	RAP
서치헬프 구현 방식	커스텀 이벤트 핸들러 클래스, 필드마다 개별 구현	CDS Value Help 선언, Search Help View 7개로 일괄 처리
화면 관리 단위	화면 9개(100/101/102/103/104/130/200/300/9000) 직접 설계	List Report + Object Page 2개, 나머지는 Facet 선언
임시저장·잠금	미구현(골격 단계)	Managed Draft 표준 액션 5개 (Edit/Activate/Discard/Resume/Prepare)
검증 로직 위치	화면 PAI 곳곳에 분산	Root BDEF 1곳에 15개 로직 집중
구조적 버그 발견 방식	화면이 뜨지 않음 — 수동 추적	프레임워크가 덤프·오류로 원인 즉시 표시

RAP이 Classic보다 쉬웠다고보다, Classic에서 직접 만들며 겪은 문제를 RAP이 프레임워크 차원에서 구조적으로 막아준다는 것을 체감했다.

담당 모듈 00~06 — 개발한 전체 프로그램

자재·공급업체·회계계정 마스터부터 구매정보레코드, 구매오더까지 7개 모듈을 하나의 데이터 모델로 통합해 개발했다. 06 구매오더관리를 대표 프로그램으로 선정해 앞서 심화 비교했다.

모듈	프로그램명	핵심 RAP 오브젝트	비고
00	공통 Search Help	CDS Search Help View 12개	전 모듈 공통 F4 지원
01	자재관리	ZTB07MARA 기반 Root/Projection View	마스터 데이터
02	FI계정관리	ZTB07SKA1 기반 Root/Projection View	FI 계정 마스터
03	밴더관리	ZTB07LFA1 기반 Root/Projection View	공급업체 마스터, Loevm 차단 로직
04	회계계정결정관리	ZTB07T030 기반 Root/Projection View	Bwart+Ktosl 기반 계정결정
05	구매정보레코드관리	ZTB07EINA/EINE 기반 Root/Projection View	PIR 헤더+조건
06	구매오더관리 대표 프로그램	ZR_B07_EKKO/EKPO, Draft 활성화	Determination 6 · Validation 5 · Instance Feature 3 · Early Numbering 1

01 검증.기본값.화면제어를 코드 곳곳에 흩어놓지 않고, **Root BDEF** 한 곳에 선언적으로 모으는 설계 방식을 익혔다.

02 Classic ABAP으로 직접 구현하며 겪은 화면 순서 의존, 수동 채번, 커스텀 서치헬프의 어려움이 **RAP이 왜 이런 방식으로 설계됐는지**를 체감하게 해줬다.

03 구매오더 대표 프로그램을 통해 Determination 6 · Validation 5 · Instance Feature 3 · Early Numbering 1, 총 15개 로직을 **트리거 시점 기준으로 구분해 설계**하는 경험을 쌓았다.

담당 모듈 00~06, 7개 프로그램을 하나의 데이터 모델로 통합한 이번 프로젝트가 **Clean Core 기반 RAP 개발의 실전 훈련** 이 되었다.